

Call

call() is a function method that **manually sets this** when invoking a function.

Calls the function immediately

Explicitly sets this

Passes arguments individually

```
function greet() {  
  console.log("Hello " + this.name);  
}
```

```
const user = { name: "Prajwal" };
```

```
greet.call(user); // Hello Prajwal
```

What happened?

Instead of:

this = global

We forced:

this = user

What Happens Internally (Conceptually)

When you do:

```
greet.call(user);
```

JavaScript roughly does this internally:

```
user.temp = greet;  
user.temp();  
delete user.temp;
```

So it temporarily attaches the function to the object,
calls it as a method,
then removes it.

That's how this becomes user.

Passing Arguments

```
function greet(age, city) {  
  console.log(this.name, age, city);  
}
```

```
greet.call(user, 23, "Mumbai");
```

Real Interview Scenario

Borrowing methods.

```
const person1 = {  
  name: "Prajwal"  
};
```

```
const person2 = {  
  name: "Rahul"  
};
```

```
function greet() {  
  console.log("Hi " + this.name);  
}
```

```
greet.call(person1); // "Hi Prajwal."  
greet.call(person2); "Hi Rahul."
```

call() vs Normal Function Call

Normal:

```
greet();
```

→ this = global / undefined

With call:

```
greet.call(user);
```

→ this = user

You override default binding.

Mental Model

Think of call() as:

“Run this function, but pretend it belongs to this object.”

Interview-Ready Definition

call() is a function method that invokes a function with an explicitly specified this value and arguments provided individually.

Apply

What is apply()?

apply() is almost the same as call().

The only difference:

👉 **Arguments are passed as an array**

```
functionName.apply(thisArg, [arg1, arg2, arg3])
```

Calls the function immediately

Sets this explicitly

Takes arguments as an array

```
function greet(age, city) {  
  console.log(this.name, age, city);  
}
```

```
const user = { name: "Prajwal" };
```

```
greet.apply(user, [23, "Mumbai"]); //Prajwal 23 Mumbai
```

apply() does NOT work on arrow functions.

🚨 Critical Limitation

Example:

```
const greet = () => {  
  console.log(this.name);  
};
```

```
greet.apply(user);
```

Still won't change this.

Because:

👉 Arrow functions do NOT have their own this.

They use lexical this.

Bind

bind() creates a **new function** where this is permanently set.

It does **NOT** call the function immediately.

1. Returns a new function
2. Locks this permanently
3. Can pre-set some arguments

```
function greet() {  
  console.log("Hello " + this.name);  
}
```

```
const user = { name: "Prajwal" };
```

```
const boundGreet = greet.bind(user);
```

```
boundGreet(); // hello Prajwal
```

Important Difference From call/apply

call/apply

```
greet.call(user); // runs immediately
```

bind

```
const fn = greet.bind(user); // does NOT run  
fn(); // runs later
```

So:

bind() prepares
call() executes

What Happens Internally (Conceptually)

When you do:

```
const boundGreet = greet.bind(user);
```

JavaScript creates a new wrapper function like:

```
function boundFunction(...args) {  
  return greet.call(user, ...args);  
}
```

So when you later call:

```
boundGreet();
```

It internally uses call() with locked this.

Why bind() Is Powerful

Because you don't lose this later.

Example (common bug):

```
const logger = {  
  prefix: "[INFO]",  
  log() {  
    console.log(this.prefix);  
  }  
};
```

`setTimeout(logger.log, 1000);` // ❌ loses this

Fix with bind:

`setTimeout(logger.log.bind(logger), 1000);` // ✅

Partial Application (Very Important)

`bind()` can also preset arguments.

```
function multiply(a, b) {  
  return a * b;  
}
```

```
const double = multiply.bind(null, 2);
```

```
console.log(double(5)); //10
```